**Министерство науки и высшего образования Российской Федерации**
**Федеральное государственное автономное образовательное учреждение высшего образования**
**«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»**
**(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

# Лабораторная работа № 3

## по курсу «Методы оптимизации»

«Метод Хука и Дживса»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Посевин Д. П.

*Москва 2026*

# 1 Задание

Необходимо выполнить:

1. поиск по направлению в случае, когда он выполняется один раз;
2. поиск по направлению в случае, когда он выполняется несколько раз и проверяется убывает ли функция;
3. сравнить количество точек в релаксационной последовательности для обоих способов приведенных выше;
4. реализовать трехмерную визуализацию.

# 2 Результаты

Исходный код программы представлен в листинге 1.

```
using Plots

rosenbrock(x) = 100 * (x[2] - x[1]^2)^2 + (x[1] - 1)^2

schwefel(x) = 418.9829 * length(x) - sum(xi -> xi * sin(sqrt(abs(xi
    ))), x)

rastrigin(x) = 10 * length(x) + sum(xi -> xi^2 - 10 * cos(2   * xi)
    , x)

function explore(f, x, h)
    n = length(x)
    x_new = copy(x)
    probes = Tuple{Vector{Float64},Bool}[]
    for i in 1:n
        f_cur = f(x_new)
        p = copy(x_new); p[i] += h[i]
        if f(p) < f_cur
            push!(probes, (p, true))
            x_new[i] += h[i]
        else
            push!(probes, (p, false))
            p2 = copy(x_new); p2[i] -= h[i]
            if f(p2) < f_cur
```

```
                push!(probes, (p2, true))
                x_new[i] -= h[i]
            else
                push!(probes, (p2, false))
            end
        end
    end
    return x_new, probes
end

function search_single(f, x0, h0, eps; alpha=2.0, beta=0.5, max_iter=10000)
    n = length(x0)
    x_base = copy(x0)
    h = isa(h0, Number) ? fill(Float64(h0), n) : copy(h0)
    traj = [copy(x_base)]
    frames = []
    iter = 0

    while maximum(abs.(h)) > eps && iter < max_iter
        x_e, probes = explore(f, x_base, h)
        dest = f(x_e) < f(x_base) ? copy(x_e) : copy(x_base)
        push!(frames, (base=copy(x_base), probes=probes, tl=length(traj), dest=dest))

        if f(x_e) < f(x_base)
            x_pat = x_e + alpha * (x_e - x_base)
            x_base = x_e
            push!(traj, copy(x_base))

            x_pe, probes2 = explore(f, x_pat, h)
            dest2 = f(x_pe) < f(x_base) ? copy(x_pe) : copy(x_base)
            push!(frames, (base=copy(x_pat), probes=probes2, tl=length(traj), dest=dest2))

            if f(x_pe) < f(x_base)
                x_base = x_pe
                push!(traj, copy(x_base))
            end
        else
            h .*= beta
        end
        iter += 1
    end

```

```
    return x_base, f(x_base), traj, frames, iter
end

function search_multi(f, x0, h0, eps; alpha=2.0, beta=0.5, max_iter
    =10000)
    n = length(x0)
    x_base = copy(x0)
    h = isa(h0, Number) ? fill(Float64(h0), n) : copy(h0)
    traj = [copy(x_base)]
    iter = 0

    while maximum(abs.(h)) > eps && iter < max_iter
        x_e, _ = explore(f, x_base, h)

        if f(x_e) < f(x_base)
            d = x_e - x_base
            x_base = x_e
            push!(traj, copy(x_base))

            while iter < max_iter
                x_pat = x_base + alpha * d
                x_pe, _ = explore(f, x_pat, h)
                if f(x_pe) < f(x_base)
                    x_prev = x_base
                    x_base = x_pe
                    d = x_base - x_prev
                    push!(traj, copy(x_base))
                    iter += 1
                else
                    break
                end
            end
        else
            h .*= beta
        end
        iter += 1
    end

    return x_base, f(x_base), traj, iter
end

function draw(f, traj, title_str, filename;
              xlims_range=nothing, ylims_range=nothing)
    xs = [p[1] for p in traj]
    ys = [p[2] for p in traj]
```

```

    if isnothing(xlims_range)
        dx = max(maximum(xs) - minimum(xs), 1.0) * 0.5
        xlims_range = (minimum(xs) - dx, maximum(xs) + dx)
    end
    if isnothing(ylims_range)
        dy = max(maximum(ys) - minimum(ys), 1.0) * 0.5
        ylims_range = (minimum(ys) - dy, maximum(ys) + dy)
    end

    x_grid = range(xlims_range[1], xlims_range[2], length=100)
    y_grid = range(ylims_range[1], ylims_range[2], length=100)
    z = [f([xi, yi]) for yi in y_grid, xi in x_grid]

    zs = [f(p) for p in traj]
    plt = surface(x_grid, y_grid, z, color=:viridis, alpha=0.5, colorbar=false, camera=(60, 30))
    plot!(plt, xs, ys, zs, color=:red, lw=2, marker=:circle, ms=3, label="")
    scatter!(plt, [xs[end]], [ys[end]], [zs[end]], color=:red, ms=6, label="", markershape=:diamond)
    xlabel!(plt, " x  "); ylabel!(plt, " x  "); zlabel!(plt, "f(x)")
    title!(plt, title_str)
    savefig(plt, filename)
    display(plt)
end

function make_gif(f, traj, frames, title_str, filename, x0, delta=1, delta2=nothing)
    n = length(frames)
    idx = n > 80 ? round.(Int, range(1, n, length=80)) : collect(1:n)

    anim = @animate for i in idx
        fr = frames[i]
        pts_x = Float64[]
        pts_y = Float64[]
        for k in 1:fr.tl
            push!(pts_x, traj[k][1]); push!(pts_y, traj[k][2])
        end
        push!(pts_x, fr.base[1]); push!(pts_y, fr.base[2])
        push!(pts_x, fr.dest[1]); push!(pts_y, fr.dest[2])
        for (p, _) in fr.probes
            push!(pts_x, p[1]); push!(pts_y, p[2])
```

```
        end

        x_min = minimum(pts_x)
        x_max = maximum(pts_x)
        y_min = minimum(pts_y)
        y_max = maximum(pts_y)
        span_x = max(x_max - x_min, 1e-10)
        span_y = max(y_max - y_min, 1e-10)
        min_half = min(span_x, span_y) < 0.1 ? 0.65 : 0.35
        pad_x = max(span_x * 0.7, min_half)
        pad_y = max(span_y * 0.7, min_half)
        xlo = x_min - pad_x
        xhi = x_max + pad_x
        ylo = y_min - pad_y
        yhi = y_max + pad_y

        x_grid = range(xlo, xhi, length=80)
        y_grid = range(ylo, yhi, length=80)
        z = [f([xi, yi]) for yi in y_grid, xi in x_grid]

        plt = contour(x_grid, y_grid, z, fill=false, color=:lightgray, lw=0.8,
                      colorbar=false, background_color=:white, foreground_color=:black)

        need_jitter = (span_x < 0.4) || (span_y < 0.4)
        view_w = xhi - xlo
        view_h = yhi - ylo
        r = need_jitter ? 0.12 * min(view_w, view_h) : 0.0

        if fr.tl > 1
            tx = [traj[k][1] for k in 1:fr.tl]
            ty = [traj[k][2] for k in 1:fr.tl]
            plot!(plt, tx, ty, color=:green, lw=2.5, label="")
            if need_jitter && fr.tl > 1
                np = fr.tl - 1
                for j in 1:np
                    off_x = r * cos(2    * (j - 1) / max(np, 1))
                    off_y = r * sin(2    * (j - 1) / max(np, 1))
                    scatter!(plt, [tx[j] + off_x], [ty[j] + off_y], color=:green, ms=4, markershape=:x, label="")
                end
            else
                scatter!(plt, tx[1:end-1], ty[1:end-1], color=:green, ms=4, markershape=:x, label="")
```

```
            end
        end

        scatter!(plt, [fr.dest[1]], [fr.dest[2]],
                 color=:green, ms=8, markershape=:x, label="")

        nprobe = length(fr.probes)
        for (j, (p, better)) in enumerate(fr.probes)
            c = better ? :blue : :red
            if need_jitter && nprobe > 0
                off_x = r * cos(2   * (j - 1) / nprobe)
                off_y = r * sin(2   * (j - 1) / nprobe)
                scatter!(plt, [p[1] + off_x], [p[2] + off_y], color=c, ms=6, markershape=:x, label="")
            else
                scatter!(plt, [p[1]], [p[2]], color=c, ms=6, markershape=:x, label="")
            end
        end

        xlabel!(plt, " x  "); ylabel!(plt, " x  ")
        title!(plt, title_str)
        xlims!(plt, xlo, xhi)
        ylims!(plt, ylo, yhi)
    end

    gif(anim, filename, fps=1.5)
end

eps = 1e-6

funcs = [("                    ", rosenbrock, [-1.5, 2.0], 0.5),
         ("                  ",  schwefel, [420.0, 420.0], 0.5),
         ("                    ", rastrigin, [3.0, 3.0], 0.5)]

results = []

for (name, f, x0, h0) in funcs
    x1, fv1, t1, fr1, it1 = search_single(f, x0, h0, eps)
    x2, fv2, t2, it2 = search_multi(f, x0, h0, eps)

    println("\n$name:")
    println("                          :")
    println("    x* = ", round.(x1, digits=6))
```

```
    println("    f(x*) = ", round(fv1, digits=10))
    println("                    : ", it1)
    println("                                    : ", length(t1))
    println("                        :")
    println("    x* = ", round.(x2, digits=6))
    println("    f(x*) = ", round(fv2, digits=10))
    println("                    : ", it2)
    println("                                    : ", length(t2))

    push!(results, (name=name, f=f, t1=t1, t2=t2, fr1=fr1))
end

println("\n                                    :")
for r in results
    println("  $(r.name):                        = $(length(r.t1)), = $(length(r.t2))")
end

draw(rosenbrock, results[1].t1, "                    ", "lab3_rosenbrock_single.png";
    xlims_range=(-2.5, 2.5), ylims_range=(-1.0, 4.0))
draw(rosenbrock, results[1].t2, "                    ", "lab3_rosenbrock_multi.png";
    xlims_range=(-2.5, 2.5), ylims_range=(-1.0, 4.0))

draw(schwefel, results[2].t1, "                    ", "lab3_schwefel_single.png")
draw(schwefel, results[2].t2, "                    ", "lab3_schwefel_multi.png")

draw(rastrigin, results[3].t1, "                    ", "lab3_rastrigin_single.png";
    xlims_range=(-1.0, 5.0), ylims_range=(-1.0, 5.0))
draw(rastrigin, results[3].t2, "                    ", "lab3_rastrigin_multi.png";
    xlims_range=(-1.0, 5.0), ylims_range=(-1.0, 5.0))

make_gif(rosenbrock, results[1].t1, results[1].fr1,
    "                    ", "lab3_rosenbrock.gif", [-1.5, 2.0], 4, 2)
```

```

make_gif(schwefel, results[2].t1, results[2].fr1,
    "                                    ", "lab3_schwefel.gif", [420.0,
     420.0], 5, 5)

make_gif(rastrigin, results[3].t1, results[3].fr1,
    "                                      ", "lab3_rastrigin.gif",
    [3.0, 3.0], 1, 0.5)
```

Результат представлен на рисунках 1– 3.

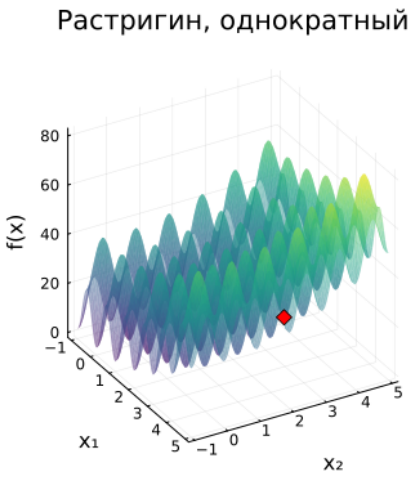


Рис. 1 — Результат

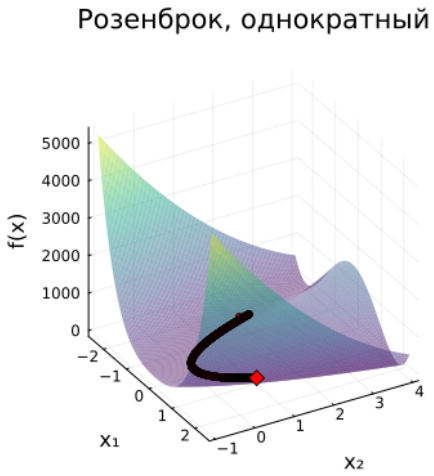


Рис. 2 — Результат

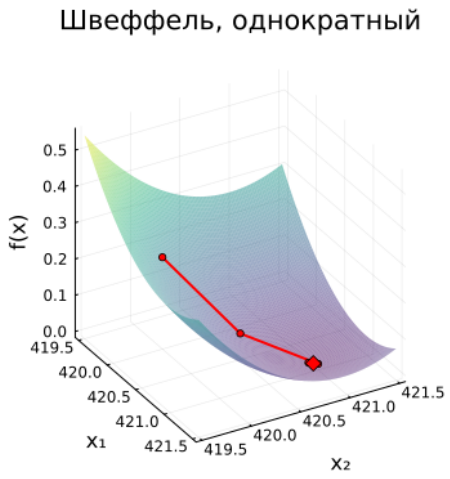


Рис. 3 — Результат